

Лабораторная работа №4

дисциплина: Архитектура компьютера

Володина Алиса Алексеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
5	Выводы	20

Список иллюстраций

4.1	Установка из коллекции репозитория	9
4.2	Установка из коллекции репозитория	9
4.3	Установка nodejs	10
4.4	pnpm setup	10
4.5	source ~/.bashrc	10
4.6	pnpm add -g commitizen	10
4.7	pnpm add -g standard-changelog	11
4.8	репозиторий git-extended	11
4.9	git commit -m "first commit, git push -u origin master	12
4.10	pnpm init и заполнение пакетов	12
4.11	git add	12
4.12	git cz	13
4.13	git push	13
4.14	git flow init	13
4.15	git branch	13
4.16	git push -all	14
4.17	git branch -set-upstream-to=origin/develop develop	14
4.18	git flow release start 1.0.0	14
4.19	standard-changelog -first-release	14
4.20	git add CHANGELOG.md, git commit -am 'chore(site): update changelog	15
4.21	git flow release finish 1.0.0	15
4.22	git push -al	15
4.23	git push -tags	15
4.24	gh release create v1.0.0 -F CHANGELOG.md	16
4.25	git flow feature start feature_branch	16
4.26	git branch	16
4.27	git flow feature finish feature_branch	17
4.28	git flow release start 1.2.3	17
4.29	Обновим номер	17
4.30	standard-changelog	18
4.31	git add CHANGELOG.md, git commit -am „chore(site): update changelog“	18
4.32	git flow release finish 1.2.3	18
4.33	git push -all	19
4.34	git push -tags	19
4.35	gh release create v1.2.3 -F CHANGELOG.md	19

Список таблиц

1 Цель работы

Получение навыков продвинутой работы с репозиторием git и релизами.

2 Задание

Выполнить работу для тестового репозитория и преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) используются для организации совместной работы коллектива над общим проектом. Как правило, основная ветка разработки хранится в репозитории — локальном или удаленном, — к которому организован доступ всех участников. Когда разработчики вносят правки, VCS позволяет регистрировать эти изменения, объединять результаты работы разных специалистов, а при необходимости — выполнять возврат к любой из более ранних версий проекта. В классической модели контроля версий применяется централизованный подход: все файлы хранятся в едином репозитории, а управление версиями обеспечивается выделенным сервером. Участник проекта перед началом работы с помощью специальных команд запрашивает актуальную или нужную ему версию файлов. Завершив внесение правок, он отправляет обновлённую версию обратно в хранилище. При этом все предыдущие версии сохраняются в центральном репозитории, и к ним можно обратиться в любой момент. Чтобы экономить дисковое пространство, сервер может не сохранять полностью каждый изменённый файл, а применять дельта-компрессию — записывать только различия между последовательными версиями. Gitflow Workflow, семантическое версионирование и Conventional Commits Gitflow Workflow — это модель ветвления для Git, которая была опубликована и популяризована Винсентом Дриссенем. Данная модель предполагает выстраивание строгой структуры веток с учётом выпуска версий проекта. Gitflow Workflow отлично подходит для организации рабочего процесса, основанного на регулярных релизах. Работа по этой модели включает создание отдельных веток для разработки новых функ-

ций, подготовки релизов, а также для срочного исправления ошибок в рабочей среде, что позволяет поддерживать стабильность основной ветки проекта на всех этапах разработки. Семантическое версионирование, или SemVer, описывается в одноимённом манифесте и представляет собой систему правил для присвоения номеров версий. Кратко его можно описать следующим образом: версия задаётся в виде кортежа МАЖОРНАЯ_ВЕРСИЯ.МИНОРНАЯ_ВЕРСИЯ.ПАТЧ. Номер версии следует увеличивать по определённым правилам. МАЖОРНУЮ версию увеличивают в том случае, когда в проект вносятся обратно несовместимые изменения API. МИНОРНУЮ версию увеличивают при добавлении новой функциональности, которая не нарушает обратной совместимости. ПАТЧ-версию увеличивают, когда выполняются обратно совместимые исправления ошибок. Дополнительные обозначения для пререлизных версий и билд-метаданных могут быть добавлены как расширение к основному формату МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ, например, в виде суффиксов -alpha, -beta или +build.15. Спецификация Conventional Commits представляет собой соглашение о том, как нужно писать сообщения коммитов. Данная спецификация полностью совместима с семантическим версионированием и даже более того — она тесно с ним связана. Conventional Commits регламентирует структуру сообщения коммита и определяет основные типы изменений, такие как feat для новой функциональности, fix для исправления ошибок, BREAKING CHANGE для изменений, нарушающих обратную совместимость, а также вспомогательные типы вроде docs, style, refactor и другие. Соблюдение этой спецификации позволяет автоматически определять следующую версию проекта в соответствии с правилами SemVer, а также генерировать changelog на основе истории коммитов, что значительно упрощает управление версиями и выпуск релизов.

4 Выполнение лабораторной работы

- 1) Установка из коллекции репозитория Copr (<https://copr.fedorainfracloud.org/coprs/elegos/gitflow>) (рис. 4.1). (рис. 4.2).

```
aavolodina@fedora:~$ sudo dnf copr enable elegos/gitflow
[sudo] пароль для aavolodina:
https://copr.fedorainfracloud.org/coprs/elegos/gitflow/ 0.0 B/s | 0.0 B | 00m01s
```

Рисунок 4.1: Установка из коллекции репозитория

```
aavolodina@fedora:~$ wget -q https://raw.githubusercontent.com/petervanderdoes/gitflow-avh/develop/contrib/gitflow-installer.sh
aavolodina@fedora:~$ ls
gitflow-installer.sh  work  Изображения  Шаблоны
quarto-1.6.40         Видео  Музыка
quarto-1.6.40-linux-amd64.tar.gz  Документы  Общедоступные
test_key.asc          Загрузки  'Рабочий стол'
aavolodina@fedora:~$ sudo bash gitflow-installer.sh
### git-flow no-make installer ###
Usage: [environment] gitflow-installer.sh [install|uninstall] [stable|develop|version] [tag]
Environment:
  PREFIX=/usr/local
  REPO_HOME=https://github.com/petervanderdoes/gitflow-avh.git
  REPO_NAME=gitflow
aavolodina@fedora:~$ sudo bash gitflow-installer.sh stable
### git-flow no-make installer ###
Usage: [environment] gitflow-installer.sh [install|uninstall] [stable|develop|version] [tag]
Environment:
  PREFIX=/usr/local
  REPO_HOME=https://github.com/petervanderdoes/gitflow-avh.git
  REPO_NAME=gitflow
aavolodina@fedora:~$
```

Рисунок 4.2: Установка из коллекции репозитория

4) Установка Node.js. (рис. 4.3).

```
aavolodina@fedora:~$ sudo dnf install nodejs
Обновление и загрузка репозитория:
Copr repo for gitflow owned by elegos      100% | 2.5 KiB/s | 2.4 KiB | 00m01s
```

Рисунок 4.3: Установка nodejs

5) Настройка Node.js Запустим pnpm setup (рис. 4.4).

```
aavolodina@fedora:~$ sudo dnf install pnpm
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет      Арх.  Версия      Репозиторий  Размер
Установка:
pnpm       noarch 10.27.0-1.fc43 updates      14.9 MiB

Сводка транзакции:
Установка:      1 пакета

Общий размер входящих пакетов составляет 3 MiB. Необходимо загрузить 3 MiB.
После этой операции будут использоваться дополнительные 15 MiB (установка 15 MiB, удаление 0 B).
Is this ok [y/N]:
```

Рисунок 4.4: pnpm setup

6) Выполним `source ~/.bashrc` (рис. 4.5).

```
aavolodina@fedora:~$ source ~/.bashrc
aavolodina@fedora:~$
```

Рисунок 4.5: `source ~/.bashrc`

7) Общие принятые коммиты

8) Выполним данную программу для помощи в форматировании коммитов. (рис. 4.6).

```
aavolodina@fedora:~$ pnpm add -g commitizen
```

Рисунок 4.6: `pnpm add -g commitizen`

9) Выполним команды для помощи в создании логов. (рис. 4.7).

```
aavolodina@fedora:~$ pnpm add -g standard-changelog
```

Рисунок 4.7: `pnpm add -g standard-changelog`

10) Создание репозитория git

Создадим репозиторий на GitHub. Для примера назовём его `git-extended`. (рис. 4.8).

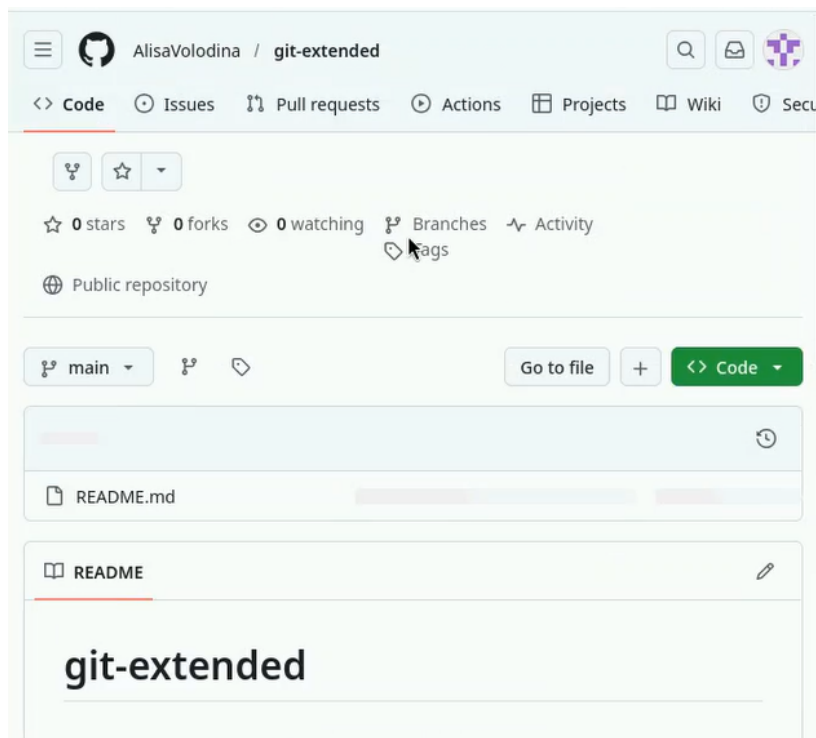


Рисунок 4.8: репозиторий `git-extended`

11) Делаем первый коммит и выкладываем на github: (рис. 4.9)

```
aavolodina@fedora:~/git-extended$ git commit -m "first commit"
[main fel0d43] first commit
1 file changed, 1 insertion(+), 1 deletion(-)
aavolodina@fedora:~/git-extended$ git remote add origin git@github.com:AlisaVolodina/git-extended.git
error: внешний репозиторий origin уже существует
aavolodina@fedora:~/git-extended$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
Запись объектов: 100% (3/3), 905 bytes | 905.00 KiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlisaVolodina/git-extended.git
2e70f28..fel0d43 main -> main
aavolodina@fedora:~/git-extended$
```

Рисунок 4.9: git commit -m "first commit", git push -u origin master

12) Конфигурация общепринятых коммитов. (рис. 4.10).

```
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:AlisaVolodina/git-extended.git",
  "author": "Alisa Volodina <1032253521@pfur.ru>",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

Рисунок 4.10: rnpm init и заполнение пакетов

13) Добавим новые файлы: (рис. 4.11).

```
aavolodina@fedora:~/git-extended$ git add .
```

Рисунок 4.11: git add .

14) Выполним коммит: (рис. 4.12).

```
aavolodina@fedora:~/git-extended$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0
```

Рисунок 4.12: git cz

15) Отправим на github: (рис. 4.13).

```
create mode 100644 package.json
aavolodina@fedora:~/git-extended$ git push
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.16 KiB | 1.16 MiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlisaVolodina/git-extended.git
fe10d43..1d22d51 main -> main
```

Рисунок 4.13: git push

16) Конфигурация git-flow

Инициализируем git-flow (рис. 4.14).

```
aavolodina@fedora:~/git-extended$ git flow init
```

Рисунок 4.14: git flow init

17) Проверьте, что Вы на ветке develop: (рис. 4.15).

```
aavolodina@fedora:~/git-extended$ git branch
* develop
main
aavolodina@fedora:~/git-extended$
```

Рисунок 4.15: git branch

12) Загрузите весь репозиторий в хранилище: (рис. 4.16).

```

aavolodina@fedora:~/git-extended$ git push --all
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/AlisaVolodina/git-extended/pull/new/develop
remote:
To github.com:AlisaVolodina/git-extended.git
 * [new branch]      develop -> develop
aavolodina@fedora:~/git-extended$

```

Рисунок 4.16: git push --all

- 13) Установите внешнюю ветку как вышестоящую для этой ветки: (рис. 4.17).

```

aavolodina@fedora:~/git-extended$ git branch --set-upstream-to=origin/develop develop
branch 'develop' set up to track 'origin/develop'.

```

Рисунок 4.17: git branch --set-upstream-to=origin/develop develop

- 14) Создадим релиз с версией 1.0.0 (рис. 4.18).

```

aavolodina@fedora:~/git-extended$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.0.0'

```

Рисунок 4.18: git flow release start 1.0.0

- 15) Создадим журнал изменений (рис. 4.19).

```

aavolodina@fedora:~/git-extended$ standard-changelog --first-release
aavolodina@fedora:~/git-extended$

```

Рисунок 4.19: standard-changelog --first-release

16) Добавим журнал изменений в индекс (рис. 4.20) (?@fig-025).

```
aavolodina@fedora:~/git-extended$ git add CHANGELOG.md
aavolodina@fedora:~/git-extended$ git commit -am 'chore(site): add changelog'
[release/1.0.0 4f85430] chore(site): add changelog
1 file changed, 1 insertion(+)
create mode 100644 CHANGELOG.md
```

Рисунок 4.20: git add CHANGELOG.md, git commit -am 'chore(site): update changelog

17) Зальём релизную ветку в основную ветку (рис. 4.21).

```
aavolodina@fedora:~/git-extended$ git flow release finish 1.0.0
```

Рисунок 4.21: git flow release finish 1.0.0

18) Отправим данные на github (рис. 4.22) (рис. 4.23).

```
aavolodina@fedora:~/git-extended$ git push --all
Перечисление объектов: 6, готово.
Подсчет объектов: 100% (6/6), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (4/4), готово.
Запись объектов: 100% (5/5), 2.67 KiB | 546.00 KiB/s, готово.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlisaVolodina/git-extended.git
1d22d51..3e26d85 develop -> develop
1d22d51..5232003 main -> main
```

Рисунок 4.22: git push -al

```
aavolodina@fedora:~/git-extended$ git push --tags
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 160 bytes | 80.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlisaVolodina/git-extended.git
* [new tag]          v1.0.0 -> v1.0.0
```

Рисунок 4.23: git push -tags

- 19) Создадим релиз на github. Для этого будем использовать утилиты работы с github: (рис. 4.24).

```
aavolodina@fedora:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/AlisaVolodina/git-extended/releases/tag/v1.0.0
```

Рисунок 4.24: gh release create v1.0.0 -F CHANGELOG.md

- 20) Работа с репозиторием git

Создадим ветку для новой функциональности: (рис. 4.25).

```
aavolodina@fedora:~/git-extended$ git flow feature start feature_branch

Переключились на новую ветку «feature/feature_branch»

Summary of actions:
- A new branch 'feature/feature_branch' was created, based on 'develop'
- You are now on branch 'feature/feature_branch'

Now, start committing on your feature. When done, use:

git flow feature finish feature_branch
```

Рисунок 4.25: git flow feature start feature_branch

- 21) Далее, продолжаем работу с git как обычно.(рис. 4.26).

```
aavolodina@fedora:~/git-extended$ git branch
develop
* feature/feature_branch
main
```

Рисунок 4.26: git branch

- 22) По окончании разработки новой функциональности следующим шагом следует объединить ветку feature_branch с develop: (рис. 4.27).


```

aavolodina@fedora:~/git-extended$ git flow feature finish feature_branch
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Уже актуально.
Ветка feature/feature_branch удалена (была 3e26d85).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
- You are now on branch 'develop'

```

Рисунок 4.27: git flow feature finish feature_branch

23) Создадим релиз с версией 1.2.3: (рис. 4.28).

```

aavolodina@fedora:~/git-extended$ git flow release start 1.2.3
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'

```

Рисунок 4.28: git flow release start 1.2.3

24) Обновите номер версии в файле package.json. Установите её в 1.2.3. (рис. 4.29).

```

{
  "name": "git-extended",
  "version": "1.2.3",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:AlisaVolodina/git-extended.git",
  "author": "Alisa Volodina <1032253521@pfur.ru>",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}

```

Рисунок 4.29: Обновим номер

25) Создадим журнал изменений (рис. 4.30).

```
aavolodina@fedora:~/git-extended$ standard-changelog
```

Рисунок 4.30: standard-changelog

26) Добавим журнал изменений в индекс (рис. 4.31).

```
aavolodina@fedora:~/git-extended$ git add CHANGELOG.md
aavolodina@fedora:~/git-extended$ git commit -am 'chore(site): update ch
angelog'
```

Рисунок 4.31: git add CHANGELOG.md, git commit -am „chore(site): update changelog“

27) Зальём релизную ветку в основную ветку (рис. 4.32).

```
aavolodina@fedora:~/git-extended$ git flow release finish 1.2.3
Переключились на ветку «main»
Эта ветка соответствует «origin/main».
Merge made by the 'ort' strategy.
CHANGELOG.md | 2 ++
package.json | 2 +-
2 files changed, 3 insertions(+), 1 deletion(-)
Уже на «main»
Ваша ветка опережает «origin/main» на 3 коммита.
(используйте «git push», чтобы опубликовать ваши локальные коммиты)
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Merge made by the 'ort' strategy.
CHANGELOG.md | 2 ++
package.json | 2 +-
2 files changed, 3 insertions(+), 1 deletion(-)
Ветка release/1.2.3 удалена (была e9529a1).

Summary of actions:
- Release branch 'release/1.2.3' has been merged into 'main'
- The release was tagged 'v1.2.3'
- Release tag 'v1.2.3' has been back-merged into 'develop'
- Release branch 'release/1.2.3' has been locally deleted
- You are now on branch 'develop'
```

Рисунок 4.32: git flow release finish 1.2.3

28) Отправим данные на github (рис. 4.33). (рис. 4.34).

```

aavolodina@fedora:~/git-extended$ git push --all
Перечисление объектов: 9, готово.
Подсчет объектов: 100% (9/9), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (6/6), готово.
Запись объектов: 100% (6/6), 2.78 KiB | 1.39 MiB/s, готово.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:AlisaVolodina/git-extended.git
   3e26d85..512e32c  develop -> develop
   5232003..63a850d  main -> main

```

Рисунок 4.33: git push --all

```

aavolodina@fedora:~/git-extended$ git push --tags
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 161 bytes | 161.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlisaVolodina/git-extended.git
 * [new tag]         v1.2.3 -> v1.2.3

```

Рисунок 4.34: git push --tags

29) Создадим релиз на github с комментарием из журнала изменений: (рис. 4.35).

```

aavolodina@fedora:~/git-extended$ gh release create v1.2.3 -F CHANGELOG.
md
https://github.com/AlisaVolodina/git-extended/releases/tag/v1.2.3
aavolodina@fedora:~/git-extended$

```

Рисунок 4.35: gh release create v1.2.3 -F CHANGELOG.md

5 Выводы

В ходе выполнения лабораторной работы я получила навыки работы с репозиториями git.